

INTERN ASSIGNMENT

/zylabs

INTERN AI ENGINEER ASSIGNMENT

Build a Production-Grade AI Research Copilot using LangGraph

“Your sellers run the conversation. We do everything else.”

Required Stack

Frontend	ReactJS
Backend	Python + FastAPI
AI Workflow	LangGraph (Mandatory)
Time Limit	2 Days

Prepared by Zylabs • [zylabs.ai](#)

Contents

Overview	3
What We Are Evaluating	3
Assignment	3
Technical Requirements	4
LangGraph Requirements	4
Architecture Requirements	5
Product & Business Thinking	5
Additional Required Documents.....	6
Submission	6

Overview

This assignment evaluates your ability to build a production-minded AI product end-to-end. We are looking for strong fundamentals, clear thinking, and the ability to ship something real. You do not need to be an expert — we want to see how you approach problems, what decisions you make, and how you communicate your reasoning.

Focus on getting one thing working well rather than everything working poorly. Quality over coverage.

What We Are Evaluating

1. Frontend Engineering
2. Backend Engineering
3. LangGraph Workflow Design
4. AI Engineering
5. Production Readiness
6. Product Thinking
7. Business Thinking

Assignment

Build an AI Research Copilot that helps a user prepare for a sales or business meeting by researching a company and generating a structured briefing.

Core User Flow

1. Create a research session using company name, website, and research objective.
2. Execute a LangGraph workflow and display progress.
3. Generate a structured research report.
4. Allow follow-up chat based on report context.
5. Persist sessions and workflow outputs.

Report Must Include

- Company Overview
- Products & Services
- Target Customers
- Business Signals

- Risks & Challenges
- Suggested Discovery Questions
- Suggested Outreach Strategy
- Unknowns
- Sources

Technical Requirements

Frontend (ReactJS)

- Research Session Creation
- Session History
- Session Detail Page
- Workflow Progress UI
- Follow-Up Chat
- Loading States
- Error States
- Responsive Design

Backend (Python + FastAPI)

- Session APIs
- Workflow Execution APIs
- Chat APIs
- Persistence Layer
- **BONUS** Error Handling
- **BONUS** Configuration Management
- **BONUS** Logging — highly recommended. Shows engineering maturity and makes debugging your own work much easier.

Illustrative Architecture Example

Frontend → Backend APIs → AI Workflow → Storage

This diagram is illustrative only. You are free to design your own architecture.

Illustrative Folder Structure

```
frontend/  
backend/  
docs/
```

README.md

LangGraph Requirements

LangGraph is mandatory. A single LLM call wrapped in an API is not acceptable.

1. Multiple meaningful nodes
2. Shared graph state
3. Conditional routing
4. Intermediate outputs
5. Failure handling
6. Recoverability

Example Workflow Shape

Planner → Research → Analysis → Quality Check → Report Generation

This is only an example. You are free to design your own workflow.

Tip: Start with a simple 2-3 node graph that works end-to-end, then add nodes. A working simple graph beats a broken complex one.

Architecture Requirements

Document your architecture in architecture.md. This is required. You do not need diagrams — clear prose is fine. Cover the following:

1. A simple diagram or ASCII sketch showing how the layers connect (frontend, backend, LangGraph, storage).
2. One short paragraph per layer: what it does and why you made that technology choice.
3. How data flows from the user's input to the final report.
4. Any notable tradeoffs or constraints you ran into.

This is not a test of technical writing. A clear, honest description of what you built is all we need.

Product & Business Thinking

Create product-improvements.md (maximum 2 pages).

Required

1. Identify at least 5 weaknesses in the current product design.
2. Prioritize the top 3 improvements you would build next.

Bonus

The bonus items below are optional but will meaningfully strengthen your submission. Attempt as many as you can within your time limit.

1. **BONUS** Explain who buys the product, who uses it, and why they would pay.
2. **BONUS** Define success metrics.
3. **BONUS** Identify the biggest cost, scaling, and reliability risks.
4. **BONUS** What feature would you remove and why?
5. **BONUS** What feature would you add and why?
6. **BONUS** If you owned this product, what would you change first and why?

Additional Required Documents

1. README.md
2. architecture.md
3. product-improvements.md

engineering-decisions.md — Bonus, Highly Recommended

This document is optional but will significantly strengthen your submission. It shows us how you think, not just what you built.

If you write it, include:

1. 3 major engineering decisions you made.
2. Alternatives you considered for each.
3. Tradeoffs you made and why.
4. **BONUS** What you would improve with 2 additional weeks.

Submission

- GitHub Repository (public or shared with us)
- README.md
- architecture.md
- product-improvements.md
- engineering-decisions.md (bonus, highly recommended)
- Demo Video or Hosted Deployment

Evaluation Rubric

Evaluation Area	Weight
Frontend Engineering	15%
Backend Engineering	20%
LangGraph Workflow Design	25%
AI Engineering	15%
Production Readiness	10%
Product & Business Thinking	15%

We evaluate interns on potential, not perfection. A well-reasoned incomplete submission outperforms a rushed but complete one. Show us how you think.